# Your First Week

Welcome to the team. I'm intentionally not asking you to "go through the project and improve it" right away — that tends to invite random, ungrounded changes from AI coding tools. Instead, here's bounded ownership to start with.

## First-Week Tasks

1. **Get oriented**
   - Read `README.md`, `SETUP.md`, `CAUSEY-DESIGN-SYSTEM.txt`, and `anti-vibecode-rules.txt`.
   - Run `npm install`, `npm test`, `npm run build`, and `npm run dev`.
   - Test `/`, `/chess`, `/event/[slug]`, `/pathways`, `/signup`, `/me`, `/family`, and `/orgs`.
   - Use mock data; don't modify `.env` or Supabase.

   *Deliverable:* send me a short list of confusing setup instructions, broken flows, and outdated documentation.

2. **Fix the outdated README**
   - `README.md` currently says accounts, auth, and admin UI don't exist — that's no longer true.
   - Update it to accurately describe the current product and routes.
   - Preserve the honest early-build language — don't claim everything is complete or verified.

   *Acceptance:* another new developer should be able to understand what currently works and run the app without help.

3. **Audit empty, error, and success states**
   - Choose one workflow: `/me`, `/family`, or `/orgs`.
   - Test its empty, loading, error, and successful states.
   - Make sure each state clearly names the next action.
   - Reuse existing components, tokens, and button patterns.

   *Acceptance:* one focused UX improvement, tests/build passing, and before/after screenshots.

4. **Add one meaningful regression test**
   - Pick a behavior you found while auditing the app.
   - Add or improve a Vitest test under `tests/`.

- Avoid tests that only check exact wording or implementation details.

*Acceptance:* show me that the test fails without the fix and passes with it.

5. **Terminology and accessibility pass**
   - Audit one complete flow for keyboard access, form labels, focus states, external-link labels, and consistent terms.
   - Keep "Causey RSVP" distinct from "organizer registration."
   - Fix only issues in that selected flow — no site-wide rewrite.

6. **Explore the scraper and auth code (read-only)**
   - Walk through the scraper architecture and the authentication/permissions code.
   - Understand how they fit together — no changes yet, just read and take notes.
   - Write up a short summary: what each piece does, what looks fragile, and any questions for me.

7. **Document `lib/format.ts`**
   - While you're in there for the test task, add brief JSDoc comments to each exported function.
   - Note any inputs/outputs that seem ambiguous or under-tested.
   - Keep it factual — describe what the code does, don't guess at intent you're unsure of.

### For Every Task

- Work only on `dev`; never touch `main`.
- Pull `origin/dev` before starting.
- Run `npm test` and `npm run build`.
- Update `.cursor/district-ux-progress.md`.
- Make one focused commit and push only to `origin/dev`.
- Ask Cursor to explain unfamiliar code before you edit it.

### Where to Start

Your best first coding task is adding `tests/format.test.ts` for `lib/format.ts`. It's bounded, low-risk, and teaches you the test setup without touching auth or data.

Current baseline: 152 passing tests across 26 files.

Before you start, run this — I already have some local changes sitting in the repo:

```
git status
```

### Longer-Term Ownership

Once the first week is done, pick one of these areas to own:

- **New-developer experience:** keep setup docs, architecture notes, and onboarding checks accurate.
- **Workflow QA:** rotate weekly through student, parent, coach, and org-admin flows; turn findings into one bounded fix at a time.
- **Regression testing:** add coverage whenever you find a real bug or confusing state.
- **Accessibility:** maintain keyboard, labeling, focus, mobile, and reduced-motion quality across one route group.
- **Data-trust research:** verify tournament/pathway facts against official sources and record citations — never invent or silently "correct" data.

### Off-Limits for Now

Supabase migrations/RLS and production environment variables are strictly off-limits until we pair on them directly. You can read and explore the scraper architecture and auth/permissions code (see above), but don't modify either yet — changes there need guided pairing first, as does anything involving student privacy.